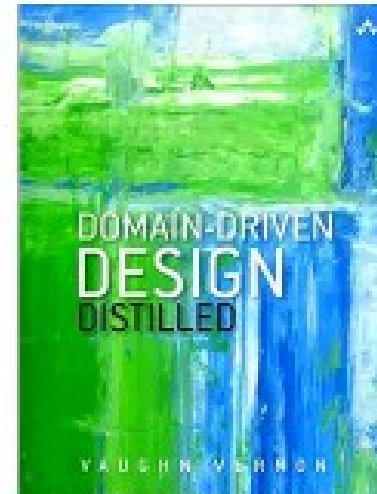
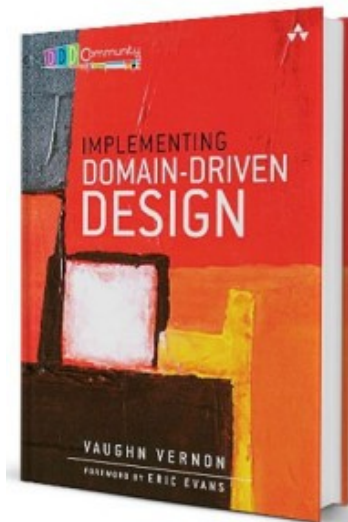


IDDD workshop



**© Copyright 2012-2020 Vaughn Vernon.
All rights reserved.**

Exercise

Value Object

- Profile Context

Doer > Skills.rankFor() : Rank

- Immutable (Side-Effect-Free Functions)
- Provide methods to calculate for:
 - Licensed, Certificate, Bonded (etc)
 - Changes **Rank** as new skills declared

NOTE: Your Domain Expert doesn't write your tests for you.

Exercise

```
public class Doer {  
    ...  
    public Rank rankFor(  
        final SkillClassification classification) {  
        return skills.rankFor(classification);  
    }  
}
```

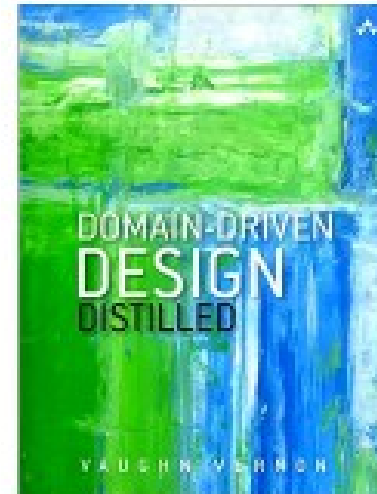
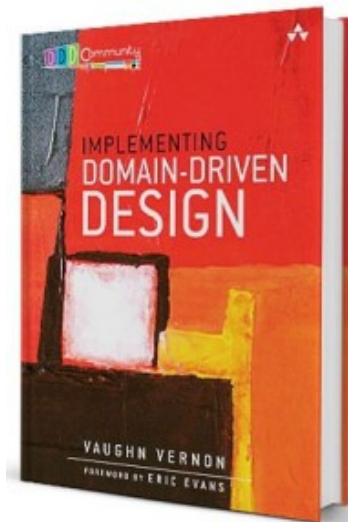
Examples

- Doer □ Skills □ Skill □ Classification;Qualifications;Rate
- Qualifications: Licensed(0.85), Certified(0.60), Bonded(0.50), Years(0.75), Specialty(0.45), Expert(0.80), Other(0.20)
- Marc has classification: #windows, #washing
- Marc has qualifications: Licensed, Certified, Bonded
- Marc's Rank: 1.95
- Daniel has classification: #windows, #washing
- Daniel has qualifications: Certified, Bonded, Years
- Daniel's Rank: 1.85
- Marc ranks higher for classification #windows, #washing

Tests

```
public void testThatMarcHasScoreOf195() {  
    Doer marc = marcDoer();  
    SkillClassification windowWashing =  
        windowWashing();  
    Rank rank = marc.rankFor(classification);  
    assertEquals(1.95, rank.total(), 0);  
}  
  
public void testThatDanielHasScoreOf185() {  
    ...  
    assertEquals(1.85, rank.total());  
}
```

IDDD workshop



**© Copyright 2012-2020 Vaughn Vernon.
All rights reserved.**

Exercise

Aggregate (Root Entity)

- Matching Context
- Implement `Proposal matchWith (Doer)`
- What is needed:
 - `ProposalMatched`
 - `Progress Spec.Matched`
 - What else?

NOTE: Your Domain Expert doesn't write your tests for you.

Examples

- Proposal ▫ EmitProposalMatched
- Need: ProposalTest
assertEquals (ProposalMatched.class,
proposal.applied().get(0).getClass())
- Need: Proposal::matchWith(Doer) { ... }
- What if already matched?
- Need: Proposal::when(ProposalMatched)
- Need: ProposalCommands::matchProposal(Id)
- Need: ProposalRepository::save(Proposal)